



Instituto Superior Politécnico de Tecnologias e Ciências
(ISPTEC)

Departamento de Engenharia e Tecnologias

Base de Dados II

Licenciatura em Engenharia Informática

2025/2026

Lab #01 – Consulta, Diagnóstico e Análise de Dados
com SQL

GRUPO N° 04

NAVEGADOR	Carlos Neves Mussagui Tchípia
------------------	-------------------------------

PILOTO	Emanuel Carneiro dos Santos
---------------	-----------------------------

TESTADOR	José Simão Tala
-----------------	-----------------

Judson Paiva
judson.paiva@isptec.co.ao

LABORATÓRIO DE SQL

Diagnóstico, Construção Progressiva e Desafio *Banco de Dados Northwind — MySQL*

Modalidade: Laboratório em Grupo	Grupo: Máx. 4 elementos	Duração: 5 dias	Consulta: Permitida
--	--------------------------------	------------------------	----------------------------

PAPÉIS DO GRUPO — rodam a cada exercício
PILOTO: digita a query no computador. | **NAVEGADOR:** lê o enunciado e orienta. | **TESTADOR:** analisa o resultado e preenche o carimbo. | **RELATOR:** regista o raciocínio e apresenta se solicitado.

FASE 1 Detecta o Erro 20 min	FASE 2 Constrói em Camadas 45 min	FASE 3 Desafio em Aberto 25 min
--	---	---

FASE 1 — DETECTA O ERRO
Cada query contém exactamente um erro. O grupo identifica o erro, redige a justificação técnica e reescreve a query correcta. A linha incorrecta está assinalada.

Competência avaliada	Indicador de desempenho
Compreensão	Explica por que a cláusula, a condição ou a estrutura está incorrecta, sem apoio externo.
Diagnóstico de erros	Distingue erros de sintaxe de erros semânticos e identifica a regra SQL violada.
Correcção técnica	Reescreve a query de forma válida e com resultado correcto, confirmado pelo carimbo.

Exercício 01 — Erro de cláusula de filtragem

A query seguinte foi escrita para listar as categorias da tabela products com faturamento bruto total (SUM de quantity × unit_price de order_details) superior a \$500. Identifique o erro, justifique e reescreva a query correcta.

```
SELECT
  p.category,
  ROUND(SUM(od.quantity * od.unit_price), 2) AS faturamento_bruto,
  COUNT(DISTINCT od.order_id) AS pedidos_distintos
FROM order_details od
JOIN products p ON od.product_id = p.id
GROUP BY p.category
WHERE SUM(od.quantity * od.unit_price) > 500
ORDER BY faturamento_bruto DESC;
```

Justificação do erro (o grupo explica por que a linha está errada):

Erro sintático. Não se pode usar a cláusula Where Para filtrar resultados de funções agregadoras, para isso utiliza-se o Having.

Query corrigida:

```
SELECT
  p.category,
  ROUND(SUM(od.quantity * od.unit_price), 2) AS faturamento_bruto,
  COUNT(DISTINCT od.order_id) AS pedidos_distintos
FROM order_details od
JOIN products p ON od.product_id = p.id
GROUP BY p.category
HAVING SUM(od.quantity * od.unit_price) > 500
ORDER BY faturamento_bruto DESC;
```

CARIMBO — Q01 *(preencher após executar)*

N.º de linhas retornadas	13
1.º valor de: faturamento_bruto	38260.25
Último valor de: faturamento_bruto	533.75

Exercício 02 — *Erro na condição de ligação entre tabelas*

A query seguinte foi escrita para listar todos os pedidos da tabela orders com o nome completo do cliente correspondente da tabela customers. O código contém um erro que provoca um resultado incorreto. Identifique o erro, justifique e reescreva a query correcta.

```
SELECT
  o.id AS pedido_id,
  CONCAT(c.first_name, ' ', c.last_name) AS cliente,
  DATE_FORMAT(o.order_date, '%d/%m/%Y') AS data_pedido,
  ROUND(o.shipping_fee, 2) AS frete
```

```
FROM orders o
JOIN customers c ON o.id = c.id
ORDER BY cliente ASC;
```

Justificação do erro:

Erro semântico, pois a relação entre as chaves no Join é entre 2 chaves primárias (nunca serão iguais), a relaç

Query corrigida:

```
JOIN customers c ON o.customer_id = c.id
```

CARIMBO — Q02 *(preencher após executar)*

N.º de linhas retornadas	48
1.º valor de: pedido_id	45
Último valor de: cliente	Thomas Axen

Exercício 03 — Erro na referência a colunas calculadas

A query seguinte foi escrita para listar os pedidos da tabela orders cujo frete (shipping_fee) está acima da média geral, utilizando uma subconsulta escalar. Ao executar, o MySQL retorna: Error Code: 1054. Unknown column 'frete' in 'where clause'. Identifique o erro, justifique e reescreva a query correcta.

```
SELECT
  id,
  customer_id,
  DATE_FORMAT(order_date, '%d/%m/%Y') AS data_pedido,
  ROUND(shipping_fee, 2) AS frete FROM
orders
WHERE frete > (SELECT AVG(shipping_fee) FROM orders)
ORDER BY frete DESC;
```

Justificação do erro:

Erro semântico, porque não existe a coluna frete na tabela orders, o uso do alias apenas renomeia o campo shipping_fee.

Query corrigida:

```
SELECT id,
customer_id,
DATE_FORMAT(order_date, '%d/%m/%Y') AS data_pedido,
ROUND(shipping_fee, 2) AS frete FROM orders

WHERE ROUND(shipping_fee, 2) > (

SELECT AVG(shipping_fee) FROM orders) ORDER BY frete DESC;
```

CARIMBO — Q03 *(preencher após executar)*

N.º de linhas retornadas	14
1.º valor de: frete	300.00
Último valor de: frete	50.00 ,

Exercício 04 — Erro na estrutura de subconsulta como fonte de dados

A query seguinte foi escrita para listar os funcionários da tabela employees ordenados pelo total de pedidos processados, utilizando uma subconsulta no FROM. Ao executar, o MySQL retorna: Error Code: 1248. Every derived table must have its own alias. Identifique o erro, justifique e reescreva a query correcta.

```
SELECT
funcionario,
total_pedidos
FROM (
    SELECT
        CONCAT(e.first_name, ' ', e.last_name) AS funcionario,
        COUNT(o.id) AS total_pedidos
    FROM employees e
    LEFT JOIN orders o ON o.employee_id = e.id
    GROUP BY e.id, funcionario
)
ORDER BY total_pedidos DESC;
```

Justificação do erro:

Toda subquery retorna uma tabela e neste caso há falta de alias para a subconsulta.

Query corrigida:

```
SELECT
funcionario,
total_pedidos
FROM (
    SELECT
        CONCAT(e.first_name, ' ', e.last_name) AS funcionario,
        COUNT(o.id) AS total_pedidos
    FROM employees e
    LEFT JOIN orders o ON o.employee_id = e.id
    GROUP BY e.id, funcionario
) as funcionarios
ORDER BY total_pedidos DESC;
```

CARIMBO — Q04 (preencher após executar)

N.º de linhas retornadas	9
1.º valor de: funcionario	Nancy Freehafer
Último valor de: total_pedidos	0

FASE 2 — CONSTRÓI EM CAMADAS

Cada exercício é dividido em passos obrigatórios. O grupo executa cada passo, valida o resultado com o carimbo e responde à reflexão escrita antes de avançar. Não é permitido saltar passos.

Competência avaliada	Indicador de desempenho
Aplicação	Constrói consultas SQL correctas a partir de um enunciado, sem modelo de solução.
Análise	Decompõe um problema complexo em etapas, valida cada etapa e identifica dependências entre cláusulas.
Raciocínio técnico escrito	Formula por escrito uma resposta técnica fundamentada sobre o comportamento da query.

Exercício 01 — Faturamento líquido por empresa cliente

Construa uma consulta que apresente o faturamento líquido total por empresa cliente (customers.company), calculado como $\text{quantity} \times \text{unit_price} \times (1 - \text{discount})$ a partir de order_details, com ligação através de orders. Devem ser exibidas apenas empresas com faturamento líquido superior a \$500, ordenadas pelo faturamento decrescente. Os passos seguintes devem ser executados e validados por ordem.

Passo 1 Ligar orders a customers e verificar a correspondência dos registos.

Execute a query fornecida e preencha o carimbo.

```
SELECT
  o.id          AS pedido_id,
  c.company     AS empresa_cliente
FROM orders o
JOIN customers c ON o.customer_id = c.id
ORDER BY empresa_cliente;
```

CARIMBO — Q2.1a (preencher após executar)

N.º de linhas retornadas	48
1.º valor de: pedido_id	44
Último valor de: empresa_cliente	Company z

Passo 2 Acrescentar order_details e calcular a receita líquida por linha de pedido.

Execute a query fornecida e preencha o carimbo.

```
SELECT
  c.company
AS empresa_cliente,    od.quantity,    od.unit_price,    od.discount,
```



```

ROUND(od.quantity * od.unit_price * (1 - od.discount), 2) AS
receita_linha FROM orders o
JOIN customers c ON o.customer_id = c.id
JOIN order_details od ON o.id = od.order_id ORDER BY
empresa_cliente;

```

CARIMBO — Q2.1b *(preencher após executar)*

N.º de linhas retornadas	58
1.º valor de: empresa_cliente	Company A
Último valor de: receita_linha	2250

Passo 3 Agrupar por empresa e calcular o total de receita líquida.

Construa o passo 3 acrescentando GROUP BY e SUM à query anterior:

```

SELECT
  c.company AS empresa_cliente,
  SUM(od.quantity),
  SUM(od.unit_price),
  SUM(od.discount),
  SUM(ROUND(od.quantity * od.unit_price * (1 - od.discount), 2)) AS receita_linha
FROM orders o
JOIN customers c ON o.customer_id = c.id
JOIN order_details od ON o.id = od.order_id
GROUP BY empresa_cliente
ORDER BY empresa_cliente;

```

CARIMBO — Q2.1c *(preencher após executar)*

N.º de linhas retornadas	15
1.º valor de: empresa_cliente	Company A
Último valor de: receita_liquida_total	3625.25

Passo 4 Filtrar empresas com faturamento superior a \$500 e ordenar o resultado.

Construa a query final acrescentando HAVING e ORDER BY:

```

SELECT
  c.company AS empresa_cliente,

```

```

SUM(od.quantity),
SUM(od.unit_price),
SUM(od.discount),
SUM(ROUND(od.quantity * od.unit_price * (1 - od.discount), 2)) AS receita_linha
FROM orders o
JOIN customers c ON o.customer_id = c.id
JOIN order_details od ON o.id = od.order_id
GROUP BY empresa_cliente
HAVING SUM(ROUND(od.quantity * od.unit_price * (1 - od.discount), 2)) > 500
ORDER BY SUM(ROUND(od.quantity * od.unit_price * (1 - od.discount), 2)) ;

```

CARIMBO — Q05 *(preencher após executar)*

N.º de linhas retornadas	15
1.º valor de: empresa_cliente	Company Y
Último valor de: receita_liquida_total	15432.5

Reflexão escrita — Se o *HAVING* fosse substituído por *WHERE* com a mesma condição, o MySQL aceitaria? O resultado seria diferente? Justifique com base na ordem de execução das cláusulas SQL.

Não, o resultado seria diferente pois resultaria em um erro, estaríamos a infringir a ordem lógica de execução estabelecida pelo MySQL, utiliza-se a cláusula *Having* para filtrar resultados de funções agregadoras permitindo fazer a filtragem de um grupo específico de elementos.

Exercício 02 — Volume de vendas por funcionário e por mês

Construa uma consulta que apresente, por funcionário (employees) e por mês/ano (formato MM/YYYY), o total de pedidos distintos processados e o faturamento bruto (SUM de quantity ×

unit_price de order_details), utilizando as tabelas employees, orders e order_details. A consulta final deve usar GROUP BY com duas colunas e ser ordenada por funcionário crescente e mês crescente.

Passo 1 Ligar employees a orders e formatar a data de pedido como mês/ano.

Execute a query fornecida e preencha o carimbo.

```
SELECT
  CONCAT(e.first_name, ' ', e.last_name) AS funcionario,
  DATE_FORMAT(o.order_date, '%m/%Y') AS mes_ano,
  o.id AS pedido_id
FROM orders o
JOIN employees e ON o.employee_id = e.id
ORDER BY funcionario, mes_ano;
```

CARIMBO — Q2.2a (preencher após executar)

N.º de linhas retornadas	48
1.º valor de: funcionario	Andrew Cencini
Último valor de: pedido_id	73

Passo 2 Acrescentar order_details e calcular o faturamento bruto por linha.

Construa o passo 2 acrescentando o JOIN com order_details e a coluna de faturamento:

```
SELECT
  CONCAT(e.first_name, ' ', e.last_name) AS funcionario,
  DATE_FORMAT(o.order_date, '%m/%Y') AS mes_ano,
  o.id AS pedido_id,
  (od.quantity * od.unit_price) AS faturamento_bruto
FROM orders o
JOIN employees e ON o.employee_id = e.id
JOIN order_details od ON o.id = od.order_id
ORDER BY funcionario, mes_ano;
```

CARIMBO — Q2.2b (preencher após executar)

N.º de linhas retornadas	58
1.º valor de: funcionario	Andrew Cencini
Último valor de: faturamento_bruto	96.50000000

Passo 3 Agrupar por funcionário e por mês com GROUP BY de duas colunas.

Construa a query final com GROUP BY, COUNT DISTINCT, SUM e ORDER BY:

```
SELECT
  CONCAT(e.first_name, ' ', e.last_name) AS funcionario,
```

```

DATE_FORMAT(o.order_date, '%m/%Y') AS mes_ano,
COUNT(DISTINCT o.id) AS pedido_id,
SUM(od.quantity * od.unit_price) AS faturamento_bruto
FROM orders o
JOIN employees e ON o.employee_id = e.id
JOIN order_details od ON o.id = od.order_id
GROUP BY funcionario, mes_ano
ORDER BY funcionario, mes_ano;

```

CARIMBO — Q06 *(preencher após executar)*

N.º de linhas retornadas	27
1.º valor de: funcionario	Andrew Cencini
Último valor de: faturamento_bruto	96.50000000

Reflexão escrita — O que aconteceria ao resultado se o GROUP BY tivesse apenas uma coluna (só funcionario, sem mes_ano)? O número de linhas aumentaria ou diminuiria? Os valores de faturamento_bruto mudariam? Justifique.

O número de linhas diminuiria, pois o agrupamento seria geral a nível do funcionário, quanto ao faturamento_bruto seria a soma do que cada funcionário faturou em todos os meses.

Exercício 03 — Produtos com preço acima da média da sua categoria

Construa uma consulta que liste os produtos activos (discontinued = 0) da tabela products cujo list_price é superior à média da sua própria category, apresentando também essa média como coluna media_categoria. A consulta final deve utilizar subconsulta correlacionada no WHERE e no SELECT. Ordenar por category crescente e list_price decrescente.

Passo 1 Calcular a média global de preço dos produtos activos.

Execute a query fornecida, registre o valor no espaço indicado e preencha o carimbo.

```

SELECT ROUND(AVG(list_price), 2) AS media_global
FROM products
WHERE discontinued = 0;

-- Valor obtido: 15.85

```

Passo 2 Filtrar produtos com preço acima da média global, usando subconsulta escalar no WHERE.

```

SELECT product_name, category, ROUND(list_price, 2) AS preco_lista
FROM products
WHERE discontinued = 0

```

```
AND list_price > (SELECT AVG(list_price) FROM products WHERE discontinued = 0)
ORDER BY category, list_price DESC;
```

CARIMBO — Q2.3a *(preencher após executar)*

N.º de linhas retornadas	18
1.º valor de: category	Baked Goods & Mixes
Último valor de: preco_lista	17.00

Passo 3 Adaptar a subconsulta para ser correlacionada com a categoria de cada produto.

Construa a query final com subconsulta correlacionada no WHERE e no SELECT (alias p1 e p2):

```
SELECT
  p1.product_name,
  p1.category,
  ROUND(p1.list_price, 2) AS preco_lista,
  (
    SELECT ROUND(AVG(p2.list_price), 2)
    FROM products p2
    WHERE p2.discontinued = 0 AND p2.category = p1.category
  ) AS media_categoria
FROM products p1
WHERE p1.discontinued = 0
AND p1.list_price > (
  SELECT AVG(p2.list_price)
  FROM products p2
  WHERE p2.discontinued = 0 AND p2.category = p1.category)
ORDER BY p1.category ASC, p1.list_price DESC;
```

CARIMBO — Q07 *(preencher após executar)*

N.º de linhas retornadas	14
--------------------------	----

1.º valor de: category	Baked Goods & Mixes
Último valor de: preco_lista	9.65

Reflexão escrita — *A subconsulta correlacionada é executada uma vez por linha da consulta exterior. Para uma tabela products com 45 linhas, quantas vezes a subconsulta é executada? Que implicação tem isso em termos de desempenho quando a tabela tiver milhões de registos?*

A subconsulta correlacionada será executada 45 vezes, uma para cada linha da consulta principal. Em tabelas com milhões de registos, isso pode causar baixo desempenho, porque a subconsulta será repetida muitas vezes, aumentando o tempo de execução da consulta.

Exercício 04 — Ranking dos 5 produtos mais vendidos

Construa uma consulta que apresente os 5 produtos mais vendidos em quantidade total (SUM de quantity de order_details), exibindo: posição no ranking, nome do produto (products.product_name), categoria e total de unidades vendidas. A posição deve ser gerada com uma variável de sessão @pos. A consulta final deve utilizar uma subconsulta como fonte de dados no FROM.

Passo 1 Calcular o total de unidades vendidas por produto e limitar aos 5 primeiros.

Execute a query fornecida e preencha o carimbo.

```
SELECT
    p.product_name,
    p.category,
    SUM(od.quantity) AS total_unidades
FROM order_details od
JOIN products p ON od.product_id = p.id
GROUP BY p.id, p.product_name, p.category
ORDER BY total_unidades DESC
LIMIT 5;
```

CARIMBO — Q2.4a (preencher após executar)

N.º de linhas retornadas	5
1.º valor de: product_name	Northwind Traders Coffee
Último valor de: total_unidades	200.0000

Passo 2 Envolver a query do passo anterior numa subconsulta no FROM e acrescentar a coluna de ranking.

Inicialize a variável @pos com SET @pos := 0 antes de executar a consulta principal.

Construa a query final com SET e @pos := @pos + 1 como coluna de posição:

```
SET @pos := 0;

SELECT
    (@pos := @pos + 1) AS posicao,
    product_name,
    category,
    total_unidades
FROM (
    SELECT
        p.product_name,
        p.category,
```

```

SUM(od.quantity) AS total_unidades
FROM order_details od
JOIN products p ON od.product_id = p.id
GROUP BY p.id, p.product_name, p.category
ORDER BY total_unidades DESC
LIMIT 5
) AS vendas;

```

CARIMBO — Q08 *(preencher após executar)*

N.º de linhas retornadas	5
1.º valor de: pos	1
Último valor de: total_unidades	200.0000

Reflexão escrita — A variável @pos é inicializada com SET @pos := 0 antes da consulta principal. O que aconteceria se a variável não fosse reinicializada e a query fosse executada uma segunda vez na mesma sessão MySQL? O ranking seria correcto? Justifique.

NÃO, o ranking não seria correcto, pois teria os valores acumulados.

FASE 3 — DESAFIO EM ABERTO

Não é fornecida qualquer query de referência. O grupo analisa o problema, define a abordagem e escreve a consulta completa de raiz. O resultado é validado com o carimbo.

Competência avaliada	Indicador de desempenho
Avaliação	Compara duas abordagens válidas, fundamenta a escolha e identifica limitações de cada uma.
Criação	Formula o problema de análise, selecciona as tabelas e constrói a consulta

	de forma autónoma.
Transferência	Aplica conceitos trabalhados nas fases anteriores a contextos e tabelas diferentes.

Exercício 01 — *Fornecedores com pedidos de compra em aberto*

Elabore uma consulta que liste os fornecedores da tabela `suppliers` que possuem pedidos de compra na tabela `purchase_orders` com `status_id` IN (1, 2), indicando para cada fornecedor: o nome da empresa (`suppliers.company`), o número de pedidos de compra em aberto e o custo total comprometido (SUM de `unit_cost` × `quantity` de `purchase_order_details`). Devem ser apresentados apenas fornecedores com mais de 1 pedido em aberto. Ordene pelo custo total decrescente.

Tabelas: `suppliers`, `purchase_orders`, `purchase_order_details`, `purchase_order_status`

Escreva a sua query aqui:

```
SELECT
    company,
    COUNT(DISTINCT po.id) numero_pedido_compra,
    SUM(pod.quantity * pod.unit_cost) custo_total
FROM
    suppliers s
JOIN
    purchase_orders po ON s.id = po.supplier_id
JOIN
    purchase_order_status pos ON pos.id = po.status_id
JOIN
    purchase_order_details pod ON pod.purchase_order_id = po.id
```

```
WHERE status_id IN (1, 2)

GROUP BY s.company

HAVING numero_pedido_compra > 1

ORDER BY custo_total DESC;
```

CARIMBO — Q09 *(preencher após executar)*

N.º de linhas retornadas	5
1.º valor de: company	Supplier A
Último valor de: custo_total	201.00000000

Exercício 02 — Análise comparativa de duas abordagens — NOT EXISTS vs NOT IN

As duas queries seguintes pretendem obter o mesmo resultado: listar os clientes da tabela customers que não possuem nenhum registo na tabela orders. Ambas correm sem erro de sintaxe. O grupo deve: (a) executar cada query e preencher os respectivos carimbos; (b) verificar se os resultados são idênticos; (c) responder por escrito às questões de análise.

ABORDAGEM A — NOT IN com subconsulta

```
SELECT
    CONCAT(first_name, ' ', last_name) AS
nome_completo,      company,      city FROM customers
WHERE id NOT IN (
    SELECT customer_id FROM orders
)
ORDER BY company ASC;
```

CARIMBO — Q3.2a (preencher após executar)

N.º de linhas retornadas	14
1.º valor de: nome_completo	Antonio Gratacos Solsona
Último valor de: company	Company X

ABORDAGEM B — NOT EXISTS com subconsulta correlacionada

```
SELECT
    CONCAT(first_name, ' ', last_name) AS
nome_completo,      company,      city
FROM customers c
WHERE NOT EXISTS (
    SELECT 1 FROM orders o WHERE o.customer_id = c.id
)
ORDER BY company ASC;
```

CARIMBO — Q3.2b (preencher após executar)

N.º de linhas retornadas	14
1.º valor de: nome_completo	Antonio Gratacos Solsona
Último valor de: company	Company X

Reflexão escrita — (i) Os dois carimbos são idênticos? Se não, qual a causa da diferença? (ii) O NOT IN pode retornar resultados inesperados se a subconsulta contiver valores NULL — explique porquê e demonstre com um exemplo conceptual. (iii) Qual das abordagens recomendariam numa base de dados de produção e porquê?

(i) Sim são idênticos

(ii) Em SQL, qualquer comparação com NULL resulta em UNKNOWN. Como a cláusula WHERE só permite passar registros que resultem em TRUE, se houver um único NULL na lista, a condição nunca será verdadeira para nenhum registro. Por exemplo, se uma subconsulta retorna os valores (1, 2, NULL). A comparação `id NOT IN (1, 2, NULL)`, equivale a `id <> 1 AND id <> 2 AND id <> NULL`, a presença do NULL torna a comparação indefinida (UNKNOWN), fazendo com que nenhum resultado seja retornado.

(iii) Em bases de dados de produção, a abordagem mais recomendada é o NOT EXISTS, porque é mais segura em relação a valores NULL e geralmente apresenta melhor desempenho e maior confiabilidade em consultas correlacionadas.

Escreva a sua query aqui:

CARIMBO — Q09 *(preencher após executar)*

N.º de linhas retornadas

1.º valor de: `company`

Último valor de: `custo_total`

Exercício 03 — Análise comparativa de transportadoras

Elabore uma consulta que apresente, para cada transportadora da tabela shippers: o nome da empresa, o total de pedidos transportados, o frete total acumulado (SUM de shipping_fee de orders) com 2 casas decimais, o frete médio por pedido (AVG de shipping_fee) com 2 casas decimais e o nome completo do funcionário (employees) que mais utilizou essa transportadora (obtido por subconsulta correlacionada: agrupamento por funcionário dentro da transportadora, ORDER BY COUNT DESC, LIMIT 1). Ordene pelo frete total decrescente. **Tabelas: shippers, orders, employees**

Escreva a sua query aqui:

```
SELECT
  company AS transportadora,
  COUNT(o.id) AS total_pedidos,
  ROUND(SUM(o.shipping_fee), 2) AS frete_total,
  ROUND(AVG(o.shipping_fee), 2) AS frete_medio,
  (
    SELECT CONCAT(e.first_name, ' ', e.last_name)
    FROM orders o2
    JOIN employees e ON o2.employee_id = e.id
    WHERE o2.shipper_id = s.id
    GROUP BY e.id
    ORDER BY COUNT(o2.id) DESC
    LIMIT 1
  ) AS nome_completo
FROM shippers s
LEFT JOIN orders o ON s.id = o.shipper_id
GROUP BY s.id, s.company
ORDER BY frete_total DESC;
```

CARIMBO — Q11 *(preencher após executar)*

N.º de linhas retornadas	3
1.º valor de: company	Shipping Company B
Último valor de: frete_medio	27.50

Exercício 04 — Consulta livre — proposta pelo grupo

O grupo formula uma pergunta de análise sobre os dados do Northwind e constrói a consulta SQL correspondente. A consulta deve incorporar obrigatoriamente: pelo menos três tabelas ligadas por JOIN, GROUP BY com duas colunas, HAVING, uma função de manipulação de datas, uma função de manipulação de texto e uma variável de sessão.

Pergunta de negócio que o nosso relatório responde:

Qual é o ranking anual de desempenho de vendas por categoria de produto e cidade do cliente, considerando apenas grupos que geraram mais de 5.000 em vendas e cujos nomes de categoria terminam com a letra 's'?

Tabelas utilizadas: products, order_details, orders e customers.

Escreva a sua query aqui:

```
SET @ranking := 0;
SELECT
  (@ranking:=@ranking + 1) AS rank_pos, tabela.*
FROM
  (SELECT
    UPPER(p.category) AS categoria,
    c.city,
    YEAR(o.order_date) AS ano_venda,
    ROUND(SUM(od.unit_price * od.quantity), 2) AS total_vendas
  FROM
    products p
  JOIN order_details od ON p.id = od.product_id
  JOIN orders o ON od.order_id = o.id
  JOIN customers c ON o.customer_id = c.id
  WHERE
    p.category LIKE '%s'
  GROUP BY ano_venda , p.category , c.city
  HAVING total_vendas > 5000
  ORDER BY total_vendas DESC) AS tabela;
```

CARIMBO — Q12 (preencher após executar)

N.º de linhas retornadas	2
1.º valor de: city	Memphis
Último valor de: total_vendas	13800.00

INVESTIGAÇÃO AUTÓNOMA

Tópicos para estudo e exploração individual ou em grupo

Indicação

Os tópicos seguintes destinam-se a aprofundamento autónomo. Não é indicado o que será avaliado nem em que formato. O objectivo é que cada aluno explore, experimente e compreenda — a profundidade do estudo reflectir-se-á na qualidade das respostas quando solicitadas.

Tópico 1 — Funções de Janela (Window Functions)

As funções ROW_NUMBER(), RANK(), DENSE_RANK(), LAG() e LEAD() permitem calcular valores em relação a outras linhas sem colapsar o resultado com GROUP BY. Investigue como diferem entre si, em que situações cada uma é preferível e como se usa a cláusula OVER() com PARTITION BY e ORDER BY.

Exemplo orientador:

```
SELECT product_name, category, list_price,  
       RANK() OVER (PARTITION BY category ORDER BY list_price DESC) AS rank_na_categoria  
FROM products  
WHERE discontinued = 0;
```

Questão de investigação: Qual a diferença entre RANK() e DENSE_RANK() quando existem empates? Escreva uma query sobre a tabela order_details que mostre, para cada produto, o seu total de unidades vendidas e a sua posição entre todos os produtos da mesma categoria.

Tópico 2 — Common Table Expressions — CTE (WITH)

A cláusula WITH permite definir tabelas temporárias nomeadas antes da consulta principal, melhorando a legibilidade e evitando subconsultas aninhadas repetidas. Investigue a sintaxe, quando é preferível a uma subconsulta no FROM e o conceito de CTE recursiva.

Exemplo orientador:

```
WITH top_clientes AS (  
  SELECT customer_id, SUM(shipping_fee) AS total_frete  
  FROM orders GROUP BY customer_id  
  ORDER BY total_frete DESC LIMIT 5  
)
```

```
SELECT c.company, tc.total_frete
FROM top_clientes tc
JOIN customers c ON tc.customer_id = c.id;
```

Questão de investigação: Reescreva o exercício E2.1 desta fase (faturamento líquido por cliente) utilizando uma CTE em vez de subconsulta no FROM. A legibilidade melhora? Justifique.

Tópico 3 — Índices e Plano de Execução (EXPLAIN)

O comando EXPLAIN mostra como o MySQL planeia executar uma query — que índices usa, quantas linhas examina e em que ordem acede às tabelas. Investigue os tipos de acesso (ALL, INDEX, REF, EQ_REF) e o que significa "Using temporary" ou "Using filesort" no output do EXPLAIN.

Exemplo orientador:

```
EXPLAIN SELECT
  p.category, SUM(od.quantity * od.unit_price) AS fat
FROM order_details od
JOIN products p ON od.product_id = p.id
GROUP BY p.category;
```

Questão de investigação: Execute EXPLAIN na query do exercício E2.2 (vendas por funcionário e mês). Quantas linhas o MySQL estima examinar? Existe algum "Using temporary"? O que isso significa em termos de desempenho?

Tópico 4 — Vistas (Views) e Consultas Reutilizáveis

Uma VIEW é uma consulta guardada na base de dados que pode ser referenciada como uma tabela. Investigue a sintaxe CREATE VIEW, quando usar uma view em vez de uma subconsulta, e as limitações das views no MySQL (nomeadamente em updates e com ORDER BY).

Exemplo orientador:

```
CREATE VIEW vw_faturamento_categoria AS
  SELECT p.category,
    ROUND(SUM(od.quantity * od.unit_price), 2) AS fat_total
  FROM order_details od
  JOIN products p ON od.product_id = p.id
  GROUP BY p.category;
```



```
-- Uso posterior:  
SELECT * FROM vw_faturamento_categoria ORDER BY fat_total DESC;
```

Questão de investigação: Crie uma view chamada vw_clientes_activos que contenha o nome completo, empresa, cidade e total de pedidos de cada cliente que tenha pelo menos 1 pedido. Escreva em seguida uma query que use essa view para listar apenas os clientes com mais de 3 pedidos.

Tópico 5 — Transacções e Integridade dos Dados

Uma transacção agrupa várias operações SQL num bloco atómico: ou todas são executadas (COMMIT) ou nenhuma é (ROLLBACK). Investigue as propriedades ACID, os comandos START TRANSACTION, COMMIT e ROLLBACK, e em que situações uma transacção é indispensável num sistema de gestão.

Exemplo orientador:

```
START TRANSACTION;  
  
UPDATE products SET discontinued = 1 WHERE id = 5;  
INSERT INTO inventory_transactions (product_id, quantity)  
VALUES (5, -10);  
-- Se ambas as operações estiverem  
correctas:  
COMMIT;  
-- Se alguma falhar: --  
ROLLBACK;
```

Questão de investigação: Num sistema de vendas, quando um pedido é confirmado, o stock do produto deve ser reduzido. Explique por que estas duas operações (INSERT em orders + UPDATE em products) devem estar dentro da mesma transacção. O que aconteceria se a segunda operação falhasse sem transacção?

REFERÊNCIAS BIBLIOGRÁFICAS

1. SILBERSCHATZ, A.; KORTH, H. F.; SUDARSHAN, S. Sistema de Banco de Dados. 7. ed. Rio de Janeiro: GEN/LTC, 2020.
2. ELMASRI, R.; NAVATHE, S. B. Sistemas de Banco de Dados. 7. ed. São Paulo: Pearson, 2019.
3. BEAULIEU, A. Learning SQL: Generate, Manipulate, and Retrieve Data. 3. ed. Sebastopol: O'Reilly, 2020.
4. SCHWARTZ, B.; ZAITSEV, P.; TKACHENKO, V. High Performance MySQL. 4. ed. Sebastopol: O'Reilly, 2022.
5. MySQL 8.0 Reference Manual. Oracle Corporation, 2024. Disponível em: <https://dev.mysql.com/doc/refman/8.0/en/>